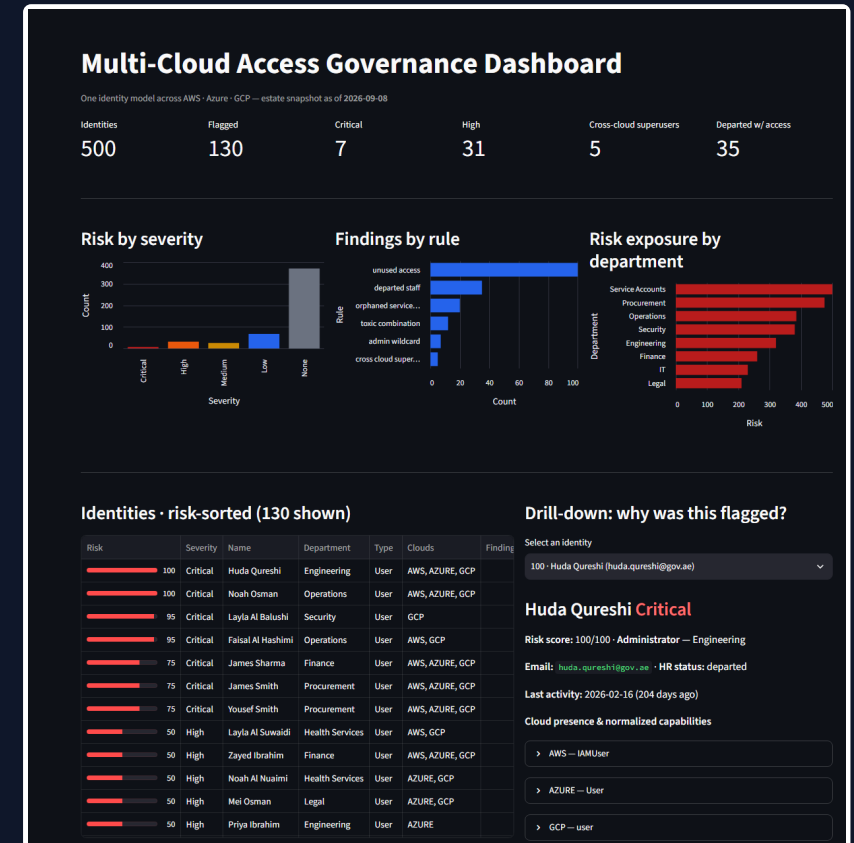


Multi-Cloud Access Governance Dashboard

One identity model across AWS, Azure and GCP: who can do what, why it is risky, and what to fix first.

COMPETITION	School of Cyber Defense 2026 - Stage 2 (Case)
CASE TOPIC	Multi-Cloud Access Governance Dashboard (difficulty: Intermediate)
TEAM	Team OPSEC: Yahya Dehbi (captain), TEAMMATE NAME 2 - fill in, TEAMMATE NAME 3 - fill in
PROTOTYPE	Python 3 + Streamlit; runs locally with one command; no cloud credentials
REPOSITORY	https://github.com/Cyber1-ops/Multi-Cloud-Access-Governance-Dashboard
SUBMITTED	11 September 2026



Live prototype on the 500-identity synthetic estate (unaltered screenshot)

The problem

Government bodies run hybrid estates across several cloud providers. Each provider has its own identity plane (AWS IAM, Azure RBAC, GCP IAM) and its own console. Over time permissions drift:

- people keep access after changing roles, and leavers are never fully deprovisioned in every cloud;
- service accounts outlive the projects that created them and keep broad rights nobody reviews;
- the same person quietly accumulates admin in all three clouds, becoming a single estate-wide blast radius;
- nobody has one view of *who can do what*, so quarterly access reviews are manual, per cloud, and late.

Objective

Build a dashboard that ingests permission data from several simulated cloud accounts in their **native formats**, normalizes them into **one common permission model**, and flags **over-privileged users, orphaned accounts and risky combinations** with a risk score and the evidence behind every finding, so a security team can act in priority order.

Who deploys it, and what it replaces

- **Owner:** the cloud-security / GRC function of a government entity (or a shared services centre serving several entities).
- **Users:** IAM engineers (remediate), risk and compliance officers (evidence for audits), department heads (attest their people's access).
- **Replaces:** manual, spreadsheet-driven quarterly access reviews and three siloed IAM consoles; complements SIEM by looking at *standing* privilege rather than events.

Minimum deliverable from the brief, and where each item lives

Brief requirement	Delivered as	Status
Synthetic but realistic IAM data, 3+ providers, different native formats	data/generate_data.py: AWS policy JSON, Azure role-assignment JSON, GCP binding JSON, plus HR directory and activity log (CSV). 500 identities, deterministic.	Done
Normalizer to a common permission model	src/common_model.py + src/normalizer.py: service:level capabilities and four escalation capabilities; explicit mapping tables per provider.	Done
Detection rules: admin/wildcard, unused access (N days), departed staff, same power across clouds, toxic combinations	src/detection.py: six rules, additive 0-100 risk score, evidence + remediation per finding, N configurable in the UI.	Done
Dashboard: identity list with risk scores, drill-down into why, filters by cloud / department	app.py (Streamlit): KPIs, charts, risk-sorted table, drill-down with per-cloud roles, normalized capabilities and findings; filters for cloud, department, severity, search, N.	Done
Exportable findings report (CSV or PDF)	src/report.py: findings CSV, identity-summary CSV and an executive PDF.	Done
Demo: 500-identity estate, sort by risk, walk through top three findings with evidence	Deterministic top three (scores 100 / 100 / 95) with full evidence trail; see pages 4-5.	Done

Success criteria we set ourselves

- Runs end-to-end from the README with one command per step, on a second machine, with no cloud accounts.
- Every score can be re-derived by hand from the rule table; every finding names its evidence.
- Second run is byte-identical; malformed or missing input degrades gracefully instead of crashing.
- Small, auditable model over a large clever one: the jury can verify it in minutes.

Architecture: ingest native formats -> resolve identities -> normalize to one model -> detect and score -> explain and export



One small, auditable permission model

Every grant, whatever its dialect, becomes a set of **service:level** capabilities over eight services (iam, storage, compute, network, database, billing, security, logging) and three levels (read, write, admin), plus four escalation capabilities: **wildcard**, **iam:admin**, **iam:create_role**, **iam:assign_role**.

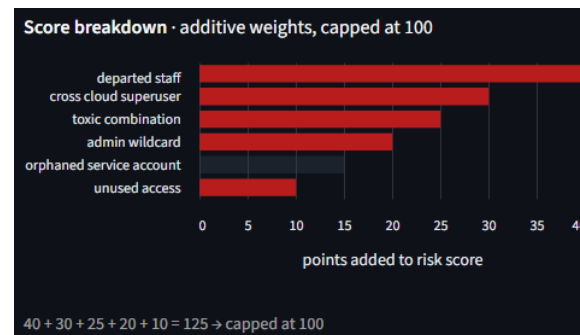
Native grant	Normalized
AWS AdministratorAccess / Azure Owner / GCP roles/owner	wildcard + admin on every service
AWS iam:CreateRole + iam:PassRole (inline policy)	iam:create_role + iam:assign_role
Azure roleAssignments/write (custom role)	iam:assign_role
GCP roles/iam.roleAdmin + serviceAccountTokenCreator	iam:create_role + iam:assign_role
AWS ReadOnlyAccess / Azure Reader / GCP roles/viewer	read on every service

Mappings are explicit tables, and conservative: when unsure the modelled privilege is raised, never lowered. A governance tool must not under-report risk.

Six rules, one explainable score

Rule	Weight
Access retained by departed employee	40
Same high privilege across all clouds	30
Privilege-escalation path (create-role + assign-role)	25
Admin / wildcard privileges	20
Orphaned / stale service account	15
Unused access (N+ days idle, default 90)	10

Score = sum of triggered weights, capped at 100. Severity is derived from the score (70+ Critical, 40+ High, 20+ Medium, above 0 Low), never asserted. Each finding carries structured evidence (which cloud, which native role, how many days idle) and a concrete remediation, so the drill-down answers *why* and *what now*.



The drill-down shows the additive score per rule (ghost bar = maximum); a capability matrix (service x cloud, colour-coded) sits directly beneath it in the app.

Why this approach, and why now

- **Standing privilege is the gap.** SIEM and CSPM see events and misconfigurations; nobody reviews who *could* act. This is the CIEM (cloud infrastructure entitlement management) problem, scoped to what a government team can run itself.
- **Zero Trust least privilege** and NIST SP 800-53 AC-2 (account management) and AC-6 (least privilege) require exactly the evidence this produces: unused access, leavers, separation of duties.
- **Multi-cloud is now the norm** for hybrid government estates; the risk lives in the seams between providers, which no single console shows.

What is distinctive

- **Cross-cloud superuser detection:** the same person holding admin-grade access in every cloud, invisible to any one provider's tooling.
- **Toxic-combination (escalation path) detection** across dialects: create-role + assign-role found whether it arrives as an AWS inline policy, an Azure custom role or two GCP roles.
- **Explainable additive scoring** a jury can re-derive by hand, with evidence and remediation on every finding. Innovation here is precision, not complexity.

Runs end-to-end from the README

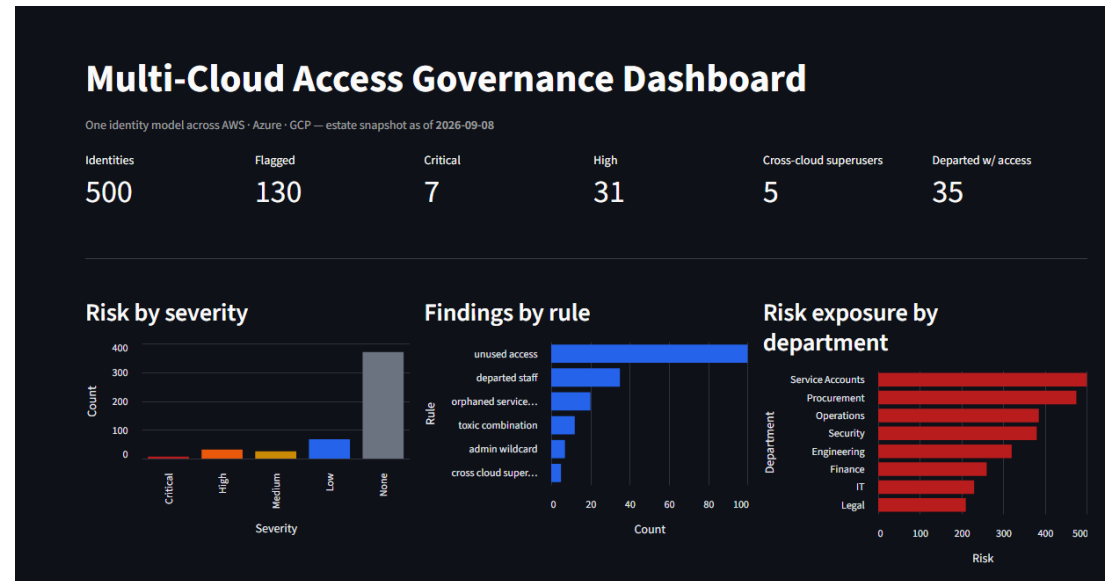
```
python -m pip install -r requirements.txt
python data/generate_data.py # deterministic 500-identity
estate
streamlit run app.py # http://localhost:8501
python -m unittest discover -s tests -v
```

What we verified, and how

- **Reproduced on a second machine** (Windows 11, Python 3.13) from a fresh clone: identical figures to the original run (500 identities, 130 flagged, 7 Critical, same top five).
- **Deterministic**: two consecutive runs of the generator are byte-identical and match the committed data; the analysis reference date is taken from the export, not the clock.
- **Dashboard exercised in the browser**: removing a cloud filter (130 -> 125 shown), free-text search (-> 6 shown, drill-down follows), unused-window slider 90 -> 150 days (flagged 130 -> 109, finding text updates), three exports downloaded. No exceptions in the server log.
- **Tested, not claimed**: 32 automated tests (stdlib unittest, about five seconds). 10 resilience cases: missing provider export, missing HR directory, missing activity log, empty directory, invalid JSON, empty CSV, non-UTF-8 file, JSON whose top level is a list or null, and wrong-typed fields everywhere (string instead of list, null roles, numeric emails, unparseable dates, unknown roles, group: and deleted: members). All degrade gracefully; the dashboard still renders. 4 dashboard cases drive the real app.py through Streamlit's AppTest (default view, slider, an identity with zero findings, an empty filter).
- **A real bug was found by these tests and fixed**: a string where a list was expected in the AWS export crashed ingestion. Every loader boundary now validates types.
- **Correctness of normalization** is asserted by 18 mapping tests (superadmin roles expand to the full wildcard set in all three dialects; the toxic inline policy yields both escalation capabilities; reader roles yield reads only; unknown roles yield nothing and stay visible as native roles).

Identity resolution across clouds

AWS keys principals by ARN, Azure by object id, GCP by *type:email* member strings. The join key is the corporate email; the activity log doubles as the bridge from GCP member strings to email. Principals present in a cloud but absent from HR are kept and labelled *Unmanaged*, which is itself a finding for service accounts.



Dashboard on the 500-identity estate: KPIs and the severity / rule / department charts (the risk-sorted identity list follows below them, see page 1).

Findings

Critical Access retained by departed employee (+40)

HR status = departed (terminated 2026-04-03) but still holds access in AWS, AZURE, GCP.

Remediation: Immediately disable/deprovision all cloud principals for this leaver.

High Same high privilege across all clouds (+30)

Admin-grade access in 3 clouds simultaneously → single identity, estate-wide blast radius. AWS: AdministratorAccess; AZURE: Owner; GCP: roles/owner

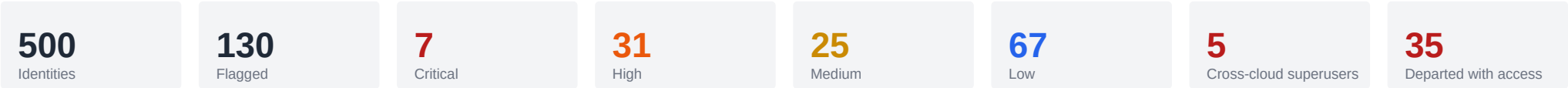
Remediation: Split duties across identities; remove blanket admin; enforce just-in-time elevation per cloud.

High Privilege-escalation path (create-role + assign-role) (+25)

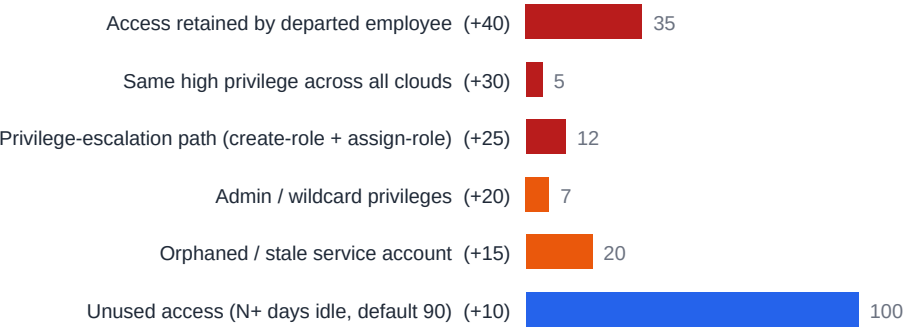
Holds both create-role and assign-role rights, allowing self-escalation to full admin. Source: AWS (AdministratorAccess); AZURE (Owner); GCP (roles/owner).

Remediation: Separate role-creation from role-assignment duties; require approval workflow for iam:PassRole / roleAssignments/write.

Drill-down for the top identity: every finding with its evidence, weight and remediation. Score 100 = 40 + 30 + 25 + 20 + 10, capped.



Findings by rule (500 identities, N = 90 days)



430 people + 70 service accounts; 359 AWS principals, 382 Azure assignments, 532 GCP binding members; 177 identities present in all three clouds.

Conclusions

- The brief's minimum deliverable is met end-to-end with a prototype a jury can run in three commands and re-derive by hand.
- Normalizing to a small common model is what makes the cross-cloud findings possible: five cross-cloud superusers and twelve escalation paths are invisible in any single console.
- Determinism and resilience were tested, not claimed; the tests found and fixed a real crash.
- Value for the owner: a prioritized, evidence-backed remediation list replaces a quarterly spreadsheet review, and the exports are the audit trail.

Top three findings, as walked through in the demo

Score	Identity	Dept	Evidence (from the drill-down)
100	Huda Qureshi	Engineering	Departed (terminated 2026-04-03) yet still AdministratorAccess (AWS), Owner (Azure), roles/owner (GCP); create-role + assign-role path; 204 days idle.
100	Noah Osman	Operations	Same pattern: leaver with admin in all three clouds, escalation path, idle.
95	Layla Al Balushi	Security	Departed; GCP roles/owner; create-role + assign-role path; idle.

All three are leavers or near-leavers holding admin-grade access with an escalation path: the exact drift the brief describes, and each is resolved by one deprovisioning action the tool names. The additive score separates them cleanly from the 100 identities flagged for idle access alone (Low).

Limitations and next steps

- **Synthetic estate.** Production would replace data/raw with exports from IAM Access Analyzer, Azure Resource Graph and gcloud get-iam-policy; only the mapping tables need extending.
- **Built-in roles only.** Unknown roles map to no capability but remain visible as native roles, so they are never silently ignored; custom-role definition parsing is the next step.
- **Single activity signal.** A production version would consume CloudTrail, Entra sign-in logs and Cloud Audit Logs directly and add group / inherited-scope resolution.
- **Next:** snapshot diffing (what changed since last review), ticket export for remediation, and per-department attestation workflows.